

REQ3 – Weather Madness

A. Weather Effect Implementation

Design 1: Implement all weather effects in one class

Pros	Cons
Simple to implement; all logic is centralized in a single class	Strong coupling between weather effects, making extension difficult
No additional abstraction, easier for beginners to understand	Each new weather effect requires modifying the class, making the system harder to extend without changing existing code.

Design 2: Use a WeatherEffect abstract class with polymorphism

Pros	Cons
Each weather effect is its own class, making the system easier to extend and maintain	If multiple weather effects have similar logic, they need to be updated manually, which is time-consuming.
New effects can be added without modifying the existing code, making the system easier to extend.	

B. Environmental System Design Comparison

Design 1: All-in-One/Direct Control in Player

Pros	Cons
Simple to implement, all logic in one place	Strong coupling between environment, weather, and player logic
Easy to trace for small projects	Adding new environment effects requires modifying core classes, making the system harder to maintain and extend.

Design 2: Decoupled EnvironmentalStatusSystem

Pros	Cons
The environmental logic is separated from weather and player, improving modularity and clarity.	Initial setup may require more effort
Easy to extend as new environment effects can be added without modifying core classes	
Promotes loose coupling and modularity	

C. Item Use Logic Design Comparison

Design 1: Separate UseTorchAction & UseUmbrellaAction Classes

Pros	Cons
The item class focuses solely on describing the item, while the action class manages its behavior.	The codebase will have more files, which may seem overwhelming at first.
Promote code reuse and modularity	Requires understanding of action patterns and class collaboration: Developers need to be familiar with how actions and items interact.
Easy to add new actions or modify existing ones without changing item classes	

Design 2: Direct Logic in Torch/Umbrella Classes

Pros	Cons
Simple and straightforward: All logic is written directly inside the item class, making it easy to see how the item behaves at a glance.	The torch/umbrella class handles too many responsibilities, making it harder to maintain and understand.

The system design adheres to the **Single Responsibility Principle SRP** by assigning each component to a focused role. In the Weather Effect implementation, an abstract WeatherEffect class and subclasses like AcidRainEffect or SnowEffect encapsulate weather-specific behavior, keeping that logic separate from the game framework. The Environmental Status System handles both player and weather logic, managing environmental changes (e.g., temperature shifts) independently. In Item Use logic, Torch and Umbrella classes maintain only item state (e.g., whether a torch is lit), while UseTorchAction and UseUmbrellaAction encapsulate usage behavior, preventing item classes from handling both state and action.

Extensibility without altering the existing code is ensured by the **Open-Closed Principle (OCP)**. New weather effects—such as a FogEffect that reduces visibility can be introduced simply by creating a new WeatherEffect subclass implementing applyEffect, with no changes to existing classes. The Environmental Status System employs an observer pattern, so additional environmental effects (e.g., wind, radiation) are added via new listener implementations rather than modifying the core system. In Item Use logic, adding a new action—say UseLanternAction for a lantern—requires creating its own class without touching existing Torch or Umbrella classes, since action logic is already encapsulated in separate action classes.

Liskov Substitution Principle (LSP) compliance is maintained by ensuring all subclasses conform to their abstract base contracts. Any WeatherEffect subclass (e.g., AcidRainEffect, SnowEffect) can substitute for WeatherEffect without altering expected behavior when applied to actors or environments. Environmental Status System listeners adhere to a common listener interface, so any listener can be interchanged without side effects. In Item Use logic, UseTorchAction and UseUmbrellaAction implement a shared Action interface, ensuring interchangeable behavior within the game's action framework.

Interface Segregation Principle (ISP) is respected by avoiding bloated interfaces. The WeatherEffect interface defines only applyEffect, so subclasses implement only necessary weather behavior. The Environmental Status System uses specific listener interfaces (e.g., weather event listeners), preventing implementers from providing unused methods. In Item Use logic, a simple Usable interface (with a single use method) ensures item classes implement only functionality relevant to usage.

Dependency Inversion Principle (DIP) is evident in how higher-level modules depend on abstractions. Game logic invokes weather effects through the WeatherEffect abstraction rather than concrete implementations. The Environmental Status System notifies listeners via an abstract interface, remaining decoupled from specific listeners. Similarly, the game engine invokes item usage through the generic Action interface rather than directly referencing UseTorchAction or UseUmbrellaAction.

The design follows the **DRY (Don't Repeat Yourself)** principle by centralizing common logic. Shared behaviors—like applying damage or buffs—reside in the `WeatherEffect` abstract class or shared utilities and are reused by all weather effect subclasses. Environmental logic is consolidated within the `EnvironmentalStatusSystem`, avoiding duplication in player or weather classes. In Item Use logic, shared action-related behavior such as updating item state or emitting messages—can be factored into a base `Action` class or utilities, reused by all action classes.

Connascence minimization reduces interdependence. `WeatherEffect` subclasses operate through a generic interface and do not require knowledge of actor internals. The `EnvironmentalStatusSystem` notifies listeners without understanding their implementations. In Item Use logic, `Torch` and `Umbrella` classes interact with action classes only via public methods (e.g., `use`, `isLit`), preventing reliance on each other's internal details.

In this design, the dominant forms of **connascence** are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**, as components such as `WeatherEffect`, `Action`, and `EnvironmentalStatusSystem` must share consistent type names and signatures to interoperate—for instance, all weather effect classes must implement `applyEffect()` with the correct parameters, and item use actions must conform to the `Action` interface. Crucially, the design avoids tighter coupling by using polymorphism and abstract interfaces, thus preventing **Connascence of Meaning (CoM)** or **Position (CoP)**, which would have arisen if shared logic were encoded in positional or implicitly understood structures. This intentional separation ensures components depend only on necessary contracts, reducing fragility and enabling smooth evolution of the system.

Maintainability and extensibility are key strengths. Introducing a new weather effect—such as `StormEffect` imposing movement penalties—requires only new `WeatherEffect` subclass. Adding a radiation zone involves creating a listener and registering it with the `EnvironmentalStatusSystem`. To add a new item like a `Lantern` (with a broader `LightAreaEffect`), implement the `Usable` interface in a new `Lantern` class and create a `UseLanternAction`; no existing code needs modification.

By adopting Design 2 for Weather Effect, Environmental System, and Item Use logic, the architecture remains flexible and scalable. Weather types can be added without altering core logic; the decoupled `EnvironmentalStatusSystem` supports modular growth; and separate action classes keep item behaviors reusable and testable. This makes the system robust in a dynamic game environment where items like `Torch` or `Umbrella` interact with weather conditions —acid rain affecting torch durability or requiring an umbrella—while remaining aligned with modern software engineering principles.

Sequence Diagram REQ3

